

Министерство образования Республики Беларусь Учреждение
образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине «ОМО»
Тема: «Сравнение классических
методов классификации»

Выполнила:
Студентка 3 курса
Группы АС-66
Петрусевич А.А.
Проверил:
Крощенко А.А.

Брест 2025

Цель работы: На практике сравнить работу нескольких алгоритмов классификации, таких как **метод k-ближайших соседей (k-NN)**, **деревья решений** и **метод опорных векторов (SVM)**. Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Ход работы

Вариант 11

Запросы:

1. Загрузить датасет по варианту; 2. Разделить данные на обучающую и тестовую выборки; 3. Обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM; 4. Для модели k-NN исследовать, как меняется качество при разном количестве соседей (k); 5. Оценить точность каждой модели на тестовой выборке;
6. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 11

- Bank Marketing • Предсказать, подпишется ли клиент на срочный вклад

- **Задания:**

1. Загрузите данные и преобразуйте категориальные признаки; 2. Разделите выборку; 3. Обучите три классификатора; 4. Сравните модели по F1-score из-за дисбаланса классов;
5. Определите, какая модель предлагает лучший компромисс для выявления потенциальных клиентов.

Код программы:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import
train_test_split, GridSearchCV
from sklearn.preprocessing import
LabelEncoder, StandardScaler
from sklearn.neighbors import
KNeighborsClassifier
from sklearn.tree import
DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import f1_score,
classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import os

print("=" * 70)
print("ЛАБОРАТОРНАЯ РАБОТА №3: Сравнение
классических методов классификации")
print("Вариант 11: Bank Marketing - прогноз
подписки на срочный вклад")
print("=" * 70)
```

```

# 1. Загрузка данных
print("\n1. ЗАГРУЗКА ДАННЫХ")
file_paths = [
    r'G:\ЛАБЫ 3
КУРС\ОМО\ml_as66\reports\Nastya\3\src\bank.csv
',
    'G:/ЛАБЫ 3
КУРС/ОМО/ml_as66/reports/Nastya/3/src/bank.csv
',
    'bank.csv',
    '../bank.csv'
]

df = None
for path in file_paths:
    try:
        df = pd.read_csv(path)
        print(f"✓ Данные загружены из:
{path}")
        break
    except:
        continue

if df is None:
    print(" Ошибка: Не удалось загрузить
данные bank.csv")
    exit(1)

# 2. Преобразование категориальных признаков
print("\n2. ПРЕОБРАЗОВАНИЕ КАТЕГОРИАЛЬНЫХ
ПРИЗНАКОВ")
print(f"Исходный размер данных: {df.shape}")

# Проверяем целевую переменную
if 'deposit' not in df.columns:
    print(" Ошибка: Отсутствует целевая
переменная 'deposit'")
    exit(1)

# Бинарное кодирование целевой переменной
df['deposit'] = df['deposit'].map({'yes': 1,
'no': 0})

# Кодирование категориальных признаков
categorical_columns = ['job', 'marital',
'education', 'default', 'housing',
'loan', 'contact',
'month', 'poutcome']
categorical_columns = [col for col in
categorical_columns if col in df.columns]

label_encoders = {}
for column in categorical_columns:
    le = LabelEncoder()
    df[column] =
le.fit_transform(df[column].astype(str))
    label_encoders[column] = le

print(f"Закодировано категориальных признаков:
{len(categorical_columns)}")

```

```

print(f"Баланс классов:
{df['deposit'].value_counts().to_dict()}")

# 3. Разделение выборки
print("\n3. РАЗДЕЛЕНИЕ ВЫБОРКИ")
X = df.drop('deposit', axis=1)
y = df['deposit']

X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.3, random_state=42,
    stratify=y
)

print(f"Обучающая выборка: {X_train.shape}")
print(f"Тестовая выборка: {X_test.shape}")

# Масштабирование для k-NN и SVM
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Обучение и оценка моделей
print("\n4. ОБУЧЕНИЕ И ОЦЕНКА МОДЕЛЕЙ")

# 4.1. K-NN с исследованием параметра k
print("\n--- K-NN: Исследование влияния
количества соседей ---")
k_range = range(1, 21)
knn_f1_scores = []

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    y_pred = knn.predict(X_test_scaled)
    f1 = f1_score(y_test, y_pred)
    knn_f1_scores.append(f1)

# Визуализация для k-NN
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(k_range, knn_f1_scores, marker='o',
linewidth=2, markersize=6)
plt.title('K-NN: Зависимость F1-score от
количества соседей', fontsize=12)
plt.xlabel('Количество соседей (k)')
plt.ylabel('F1-score')
plt.grid(True, alpha=0.3)
plt.xticks(range(1, 21, 2))

best_k = k_range[np.argmax(knn_f1_scores)]
plt.axvline(x=best_k, color='red',
linestyle='--', alpha=0.7,
label=f'Лучшее k = {best_k}')
plt.legend()

# Обучение лучшей модели K-NN
knn_best =
KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train_scaled, y_train)

```

```

y_pred_knn = knn_best.predict(X_test_scaled)
f1_knn = f1_score(y_test, y_pred_knn)

print(f"Оптимальное k: {best_k}")
print(f"F1-score K-NN: {f1_knn:.4f}")

# 4.2. Дерево решений
print("\n--- Дерево решений ---")
dt = DecisionTreeClassifier(random_state=42)
param_grid_dt = {'max_depth': range(1, 21)}
grid_dt = GridSearchCV(dt, param_grid_dt,
cv=5, scoring='f1')
grid_dt.fit(X_train, y_train)

dt_best = grid_dt.best_estimator_
y_pred_dt = dt_best.predict(X_test)
f1_dt = f1_score(y_test, y_pred_dt)

print(f"Оптимальная глубина:
{grid_dt.best_params_['max_depth']}")
print(f"F1-score Decision Tree: {f1_dt:.4f}")

# 4.3. SVM
print("\n--- Метод опорных векторов (SVM) ---")
svm = SVC(random_state=42)
param_grid_svm = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf']
}
grid_svm = GridSearchCV(svm, param_grid_svm,
cv=5, scoring='f1')
grid_svm.fit(X_train_scaled, y_train)

svm_best = grid_svm.best_estimator_
y_pred_svm = svm_best.predict(X_test_scaled)
f1_svm = f1_score(y_test, y_pred_svm)

print(f"Лучшие параметры SVM:
{grid_svm.best_params_}")
print(f"F1-score SVM: {f1_svm:.4f}")

# 5. Сравнение моделей по F1-score
print("\n5. СРАВНЕНИЕ МОДЕЛЕЙ ПО F1-SCORE")
results = {
    'K-NN': f1_knn,
    'Decision Tree': f1_dt,
    'SVM': f1_svm
}

# Визуализация сравнения
plt.subplot(1, 2, 2)
models = list(results.keys())
scores = list(results.values())
colors = ['skyblue', 'lightgreen',
'lightcoral']

bars = plt.bar(models, scores, color=colors,
edgecolor='black', alpha=0.8)
plt.title('Сравнение моделей по F1-score',
fontsize=12)

```

```

plt.ylabel('F1-score')
plt.ylim(0, 1)

# Добавляем значения на столбцы
for bar, score in zip(bars, scores):
    plt.text(bar.get_x() + bar.get_width()/2,
             bar.get_height() + 0.01,
             f'{score:.4f}', ha='center',
             va='bottom', fontweight='bold')

plt.tight_layout()
plt.savefig('models_comparison.png', dpi=300,
           bbox_inches='tight')
plt.show()

# Вывод результатов
print("\n" + "="*70)
print("РЕЗУЛЬТАТЫ:")
for model, score in results.items():
    print(f"{model:15} | F1-score: {score:.4f}")

best_model = max(results, key=results.get)
best_score = results[best_model]
print(f"\n ЛУЧШАЯ МОДЕЛЬ: {best_model} (F1-score: {best_score:.4f})")

# 6. Детальный анализ лучшей модели
print("\n6. ДЕТАЛЬНЫЙ АНАЛИЗ ЛУЧШЕЙ МОДЕЛИ")
if best_model == 'K-NN':
    print(classification_report(y_test,
                                y_pred_knn))
    best_predictions = y_pred_knn
elif best_model == 'Decision Tree':
    print(classification_report(y_test,
                                y_pred_dt))
    best_predictions = y_pred_dt
else:
    print(classification_report(y_test,
                                y_pred_svm))
    best_predictions = y_pred_svm

# Матрица ошибок для лучшей модели
plt.figure(figsize=(8, 6))
cm = confusion_matrix(y_test,
                      best_predictions)
sns.heatmap(cm, annot=True, fmt='d',
            cmap='Blues',
            xticklabels=['Не подпишется',
                          'Подпишется'],
            yticklabels=['Не подпишется',
                          'Подпишется'])
plt.title(f'Матрица ошибок: {best_model}\n',
         fontsize=14)
plt.xlabel('Предсказанный класс')
plt.ylabel('Истинный класс')
plt.savefig('confusion_matrix.png', dpi=300,
           bbox_inches='tight')
plt.show()

# 7. ВЫВОДЫ И РЕКОМЕНДАЦИИ

```

```
print("\n" + "="*70)
print("ВЫВОДЫ И РЕКОМЕНДАЦИИ ДЛЯ БАНКОВСКОГО
МАРКЕТИНГА")
print("="*70)
```

print(f"""
На основе проведенного анализа для задачи
прогнозирования подписки на срочный вклад:

```
1. ЛУЧШАЯ МОДЕЛЬ: {best_model}
   - F1-score: {best_score:.4f}
   - Наиболее эффективна для выявления
потенциальных клиентов

2. СРАВНЕНИЕ МЕТОДОВ:
   - K-NN:      {f1_knn:.4f} (лучший k =
{best_k})
   - Дерево:    {f1_dt:.4f} (глубина =
{grid_dt.best_params_['max_depth']})
   - SVM:       {f1_svm:.4f} (параметры:
{grid_svm.best_params_})
```

""")**Результат:**

```
1. ЛУЧШАЯ МОДЕЛЬ: Decision Tree
   - F1-score: 0.8159
   - Наиболее эффективна для выявления потенциальных
клиентов
```

```
2. СРАВНЕНИЕ МЕТОДОВ:
   - K-NN:    0.7663 (лучший k = 5)
   - Дерево:  0.8159 (глубина = 8)
   - SVM:     0.8101 (параметры: {'C': 1, 'kernel': 'rbf'})
```

6. ДЕТАЛЬНЫЙ АНАЛИЗ ЛУЧШЕЙ МОДЕЛИ

	precision	recall	f1-score	support
0	0.85	0.79	0.82	1762
1	0.79	0.85	0.82	1587
accuracy			0.82	3349
macro avg	0.82	0.82	0.82	3349
weighted avg	0.82	0.82	0.82	3349

5. СРАВНЕНИЕ МОДЕЛЕЙ ПО F1-SCORE

=====

=====

РЕЗУЛЬТАТЫ:

K-NN	F1-score: 0.7663
Decision Tree	F1-score: 0.8159
SVM	F1-score: 0.8101

ЛУЧШАЯ МОДЕЛЬ: Decision Tree (F1-score: 0.8159)

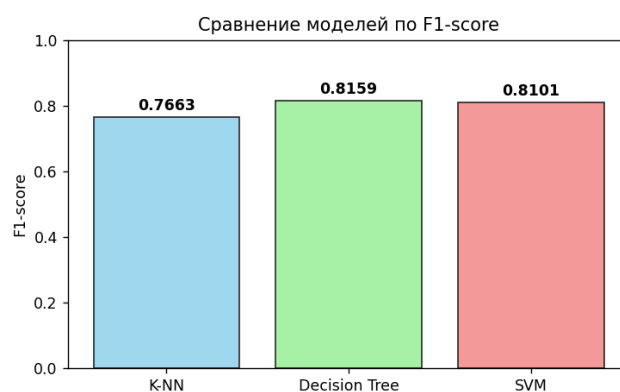
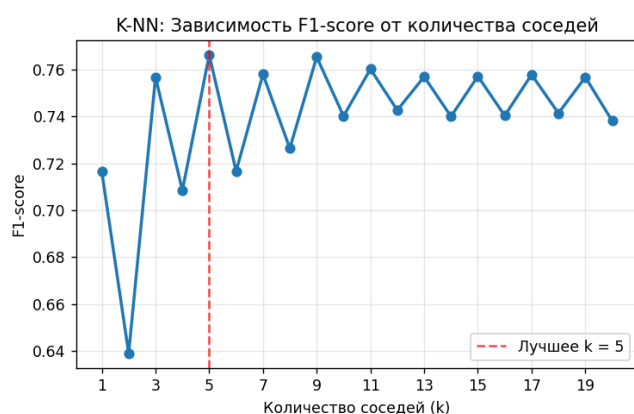
Оптимальное k: 5
F1-score K-NN: 0.7663

--- Дерево решений ---
Оптимальная глубина: 8
F1-score Decision Tree: 0.8159

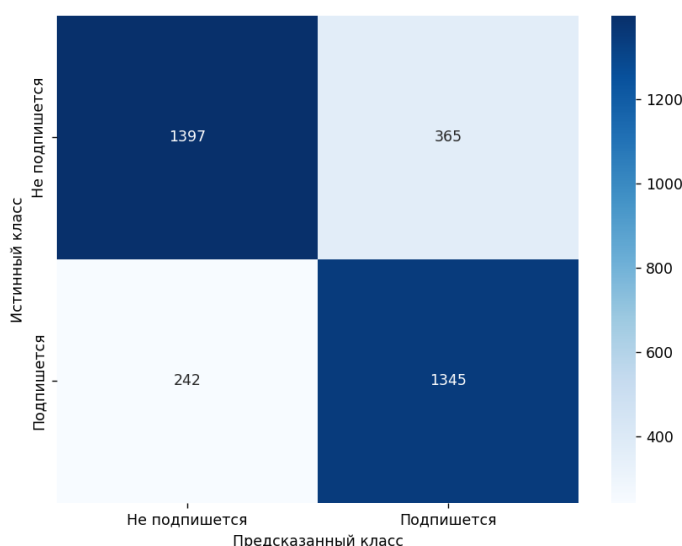
--- Метод опорных векторов (SVM) ---
Лучшие параметры SVM: {'C': 1, 'kernel': 'rbf'}
F1-score SVM: 0.8101

2. ПРЕОБРАЗОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ
Исходный размер данных: (11162, 17)
Закодировано категориальных признаков: 9
Баланс классов: {0: 5873, 1: 5289}

3. РАЗДЕЛЕНИЕ ВЫБОРКИ
Обучающая выборка: (7813, 16)
Тестовая выборка: (3349, 16)



Матрица ошибок: Decision Tree



Вывод: сравнила работу нескольких алгоритмов классификации, таких как метод **k-ближайших соседей (k-NN)**, **деревья решений** и метод **опорных векторов (SVM)**. Научилась подбирать гиперпараметры моделей и оценивать их влияние на результат.